TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Trabajo Práctico Semana 2: Testing Práctico y Contratos

Objetivo: Implementar técnicas de prueba de grado industrial y asegurar el aislamiento de componentes en el laboratorio.

1. **Implementación del Patrón AAA:** Toma una función lógica de tu sistema y escribe una suite de pruebas unitarias que siga rigurosamente las tres etapas: *Arrange* (configuración), *Act* (ejecución) y *Assert* (aseveración).

Se toma una función lógica de negocio: el cálculo del total de una compra con descuentos acumulables. Aplica 10% para clientes frecuentes y 5% adicional cuando el subtotal alcanza o supera $10.000. Rechaza subtotales negativos.

La función recibe el subtotal de la compra y un indicador de cliente frecuente, y devuelve el total a pagar. Las reglas que se verifican son:

- Si el subtotal es negativo, la operación se rechaza con una excepción controlada y el mensaje "El subtotal no puede ser negativo".
- Si el cliente es frecuente, se aplica un 10% de descuento.
- Si el subtotal es igual o mayor a $10.000, se aplica un 5% adicional (el valor límite sí incluye el descuento).
- Los descuentos se acumulan y se calculan sobre el subtotal.

Prueba	Arrange (configuración)	Act (ejecución)	Assert (aseveración)
P1. Cliente común, compra chica	Subtotal de $5.000; cliente no frecuente	Calcular el total	Aseveración de igualdad: el total es $5.000 (sin descuento)
P2. Cliente frecuente, compra chica	Subtotal de $5.000; cliente frecuente	Calcular el total	Igualdad: el total es $4.500 (10% de descuento)
P3. Cliente común, compra grande	Subtotal de $20.000; cliente no frecuente	Calcular el total	Igualdad: el total es $19.000 (5% de descuento)
P4. Cliente frecuente, compra grande	Subtotal de $20.000; cliente frecuente	Calcular el total	Igualdad: el total es $17.000 (15% acumulado)
P5. Subtotal exactamente en el umbral	Subtotal de $10.000; cliente no frecuente	Calcular el total	Igualdad: el total es $9.500, porque el límite incluye el 5%
P6. Subtotal justo por debajo del umbral	Subtotal de $9.999,99; cliente no frecuente	Calcular el total	Igualdad: el total es $9.999,99, sin descuento por un centavo de diferencia
P7. Subtotal cero	Subtotal de $0; cliente frecuente	Calcular el total	Igualdad: el total es $0 y no se produce ningún resultado inesperado
P8. Subtotal negativo (error controlado)	Subtotal de -$1; cliente no frecuente	Intentar calcular el total	Aseveración de excepción: se lanza una excepción de argumento inválido con el mensaje "El subtotal no puede ser negativo"

2. **Diseño de Dobles de Prueba:** Ante un escenario donde tu sistema debe interactuar con una API externa de pagos, redacta un informe breve explicando cuándo es necesario implementar un *Stub* (para inyección de datos indirectos) y cuándo es indispensable un *Mock* (para auditar protocolos de interacción), justificando la diferencia según la arquitectura de aislamiento.

En ecosistemas modernos, interactuar con dependencias externas como una API de pagos convierte a una prueba unitaria en una prueba de integración inestable, por lo que se requiere el uso de Dobles de Prueba para asegurar el aislamiento atómico absoluto del componente.

- Implementación de un Stub: Es necesario utilizar un Stub cuando se requiere un simulador de estado que provea entradas indirectas al componente. En el caso de la API de pagos, el Stub carece de lógica compleja y se utiliza para devolver respuestas preconfiguradas o "enlatadas" (por ejemplo, simulando una respuesta de transacción aprobada o de fondos insuficientes), permitiendo evaluar cómo reacciona nuestra aplicación ante dichos escenarios sin interactuar físicamente con la pasarela real.
- Implementación de un Mock: Por otro lado, es indispensable implementar un Mock cuando el objetivo de la prueba no es solo recibir datos, sino auditar el comportamiento y el protocolo de interacción del sistema. El Mock registrará de manera activa qué métodos invocó nuestro componente de pagos, con qué argumentos técnicos específicos, en qué orden y cuántas veces lo hizo.
- Justificación de Arquitectura: La principal diferencia según la arquitectura de aislamiento radica en que el Stub permite comprobar el estado de nuestra aplicación aislando la infraestructura, mientras que el Mock permite que la aseveración final interrogue a la dependencia simulada para garantizar que nuestra aplicación respeta estrictamente las reglas de comunicación predefinidas con la entidad externa.

3. **Análisis de Cobertura:** Dado un segmento de código que contiene una estructura condicional compleja, explica por qué el *Branch Coverage* (cobertura de ramas) ofrece una validación más rigurosa que el *Statement Coverage* (cobertura de sentencias). Diseña los casos de prueba necesarios para alcanzar el 100% de cobertura de ramas en dicho segmento.

La métrica de Statement Coverage (Cobertura de sentencias) únicamente calcula la proporción de líneas ejecutables que fueron recorridas al menos una vez, lo cual no garantiza la ausencia de fallos ante casos de borde. El Branch Coverage (Cobertura de ramas) ofrece una validación significativamente más rigurosa y segura porque no evalúa líneas físicas, sino que analiza todos los caminos condicionales y bifurcaciones lógicas posibles de la estructura de control.

Para lograr el 100% de cobertura de ramas en una estructura condicional compleja que, por ejemplo, exige que un usuario sea mayor de edad y tenga cuenta activa para acceder, se deben diseñar los siguientes casos de prueba:

- Un caso de prueba que fuerce a ambas condiciones a ser verdaderas, certificando que el flujo del programa recorre la ruta de acceso exitoso.
- Un caso de prueba donde el usuario no sea mayor de edad, obligando al sistema a transitar por la ruta alternativa o rama falsa provocada por la primera condición.
- Un caso de prueba donde el usuario sea mayor de edad pero su cuenta no esté activa, forzando al sistema a derivar hacia la rama falsa debido al segundo operando, certificando así que todas las rutas lógicas han sido validadas.